

Risk-Aware Safe Control for Decentralized Multi-Agent Systems: A Reproduction Study

Connor Hill

Department of Computer Science and Engineering

Texas A&M University

Email: crh2000@tamu.edu

Abstract—The report focuses on the reproduction of the controller framework proposed in the paper “Risk-Aware Safe Control for Decentralized Multi-Agent Systems” by implementing the algorithm’s core functions into code. These functions include Conditional Value at Risk, Control Barrier Functions, Pairwise Safety Loss, Aggregated Risk, Dynamic Responsibility Allocation, and the Decentralized Minimum-Deviation Controller. Once all methods and functions were constructed in Python code, my reconstruction of the algorithm was tested using the experimental situations of the paper: Scenario 1 and Scenario 2. Scenario 1 involves three simulated vehicles being placed on a road and needing to move past a ramp merging point quickly without any collisions, after which I recorded several metrics including the vehicles’ pairwise distances, trajectories, and aggregated risks. Scenario 2 involves six agents being tasked with swapping positions without any collisions, after which I recorded metrics including minimum pairwise distance, number of safety violations, and final goal errors per agent. I ran this test on both the decentralized risk-aware controller proposed in the paper and a non-risk-aware controller as a baseline. When testing my code for Scenario 1, the results showed that the controller was able to achieve a safety-check success rate of 99.8667 percent. As for Scenario 2, the results of my code showcased that there was a tradeoff to using the risk-aware decentralized safety controller versus the baseline non-risk-aware controller: the risk-aware system took longer to achieve its task but its agents maintained a safer distance than the baseline model, indicating slower performance but superior adherence to safety constraints.

Index Terms—Risk-aware control, decentralized multi-agent systems, control barrier functions, dynamic responsibility allocation, Conditional Value at Risk, safe control

I. INTRODUCTION

The problem with multi-agent systems is that we want all agents in the system to safely coordinate with each other as a group in order for each agent to achieve its goal. One possible solution is to use a centralized system, where there is a single global controller giving instructions to all agents at once. However, the issue with this approach is that it is computationally expensive as the more agents are added to the system, the more the system has to calculate. Another issue is that if the central controller were to malfunction, the entire system would cease to function. The paper titled “Risk-Aware Safe Control for Decentralized Multi-Agent Systems” authored by Yiwei Lyu, Wenhao Luo, and John M. Dolan, proposes using a new decentralized risk-aware safety control framework where each agent possesses its own local controller. Each local controller would be responsible for assessing their agent’s risk level, given its position relative to the other agents, allocating

the agent’s share of responsibility based on said risk level, solving the agent’s minimum-deviation safe control problem in the form of quadratic programming, moving the agent based on the safest control input produced by the safe control problem mentioned previously, and then repeating the process until the agent’s goal state has been reached.

The purpose of my project is to implement the safe control framework proposed in this paper into code and to attempt to reproduce the paper’s results. I first implemented the core components of the risk-aware decentralized safe-control framework such as Conditional Value at Risk, the pairwise Control Barrier Function, the pairwise safety loss, the aggregated agent risk, the dynamic responsibility allocation, and the minimum-deviation safe control quadratic program. After constructing these key functions, I integrated them into simulation pipelines in order to replicate the simulations for the two scenarios listed in the paper: the ramp-merging scenario and the multi-agent position swapping scenario.

II. RELATED WORK

The primary method reproduced in this project is the risk-aware decentralized safe-control framework proposed by Lyu, Luo, and Dolan. While reviewing related implementation resources, I did not find a complete official codebase for the exact dynamic responsibility allocation algorithm from the target paper. However, several public codebases implement closely related safe multi-agent control methods. The GCBF+ repository provides implementations of multiple multi-agent safety algorithms, including centralized CBF-QP and decentralized CBF-QP baselines, making it relevant to this project because my implementation also formulates safety as a quadratic program with CBF-style constraints [2]. The MACBF work studies decentralized neural barrier certificates for safe multi-agent control, where agents avoid collisions while reaching goals in a decentralized framework [3]. Cheng et al.’s robust CBF implementation is also related because it simulates multi-agent collision avoidance under uncertainty using robust Control Barrier Functions [4]. Compared with these available implementations, this project focuses specifically on reproducing the CVaR-based risk evaluation and dynamic responsibility allocation mechanism from the target paper. Therefore, the implementation was built directly from the equations and algorithmic description of the paper rather than adapted from an existing official repository.

III. METHODOLOGY

The following are the key components of the decentralized risk-aware safe control algorithm that I needed to incorporate from the paper:

A. Control Barrier Functions

The first key component of the algorithm was the Control Barrier Function represented by Eq. (1).

$$h_{ij}(x) = \|x_i - x_j\|^2 - R_{safe}^2. \quad (1)$$

This equation is one of the safety constraints that the controller needs to adhere to. The CBF is responsible for verifying whether or not two agents, i and j , are at a safe distance or not given their current positions and the minimum safe distance threshold R_{safe} .

B. Conditional Value at Risk

The next important component is CVaR (Conditional Value at Risk) represented by Eq. (2).

$$\text{CVaR}_\alpha(X) = \min_{z \in \mathbb{R}} \mathbb{E} \left[z + \frac{(X - z)^+}{1 - \alpha} \right]. \quad (2)$$

CVaR is used to measure how bad motion uncertainty can be in the worst-case tail given a confidence level alpha. For instance, if alpha = 0.95, then CVaR would return how badly uncertainty would affect safety in the worst 5 percent of the outcomes.

C. Pairwise Safety Loss

We then must incorporate Eq. (1) and Eq. (2) to determine the pairwise safety loss represented by Eq. (3).

$$\begin{aligned} L_{ij}(x, u) = & -2(x_i - x_j)^T(u_i - u_j) \\ & - 2\text{CVaR}_\alpha((x_i - x_j)^T(\epsilon_i - \epsilon_j)) \\ & - \gamma h_{ij}(x) + c. \end{aligned} \quad (3)$$

The pairwise safety loss, given the state positions and control inputs of agents i and j , measures how much safety would be lost if agents i and j were to interact.

D. Aggregated Risk

For each agent, we take the total sum of all pairwise safety losses between agent i and every other agent in the system to obtain the agent's aggregated risk, Eq. (4).

$$R_i = \sum_{j=1, j \neq i}^N L_{ij}(x, u). \quad (4)$$

E. Dynamic Responsibility Allocation

In order to determine how much responsibility to allocate to an agent i , with respect to an additional agent j , we need to divide the aggregated risk of agent j by the sum of agent j 's risk and agent i 's risk as shown in Eq. (5).

$$\omega_i = \frac{R_j}{R_i + R_j}. \quad (5)$$

An important note to make is that an agent that has higher risk will be given less responsibility as it will be more tightly constrained by the safety constraints.

F. Decentralized Minimum-Deviation Controller

The final component of the framework, and most difficult to integrate, is the decentralized minimum-deviation controller represented by Eq. (6).

$$\begin{aligned} \min_{u_i} \quad & \|u_i - \bar{u}_i\|^2 \\ \text{s.t.} \quad & u_{min} \leq u_i \leq u_{max}, \\ & A_i u_i \leq \omega_i b_i. \end{aligned} \quad (6)$$

$$A_i = -2(x_i - x_j)^T. \quad (7)$$

$$b_i = \gamma h_{ij}(x) + 2\text{CVaR}_\alpha((x_i - x_j)^T(\epsilon_i - \epsilon_j)). \quad (8)$$

The purpose of this phase of the algorithm is to determine the safest control input that satisfies the safety constraints in Eq. (6), while deviating as little as possible from the agent's nominal input. In this controller, u_i represents the optimized safe control input for agent i , while $\bar{u}_i$ represents the nominal control input that the agent would take without the safety correction. The terms A_i and b_i come from rewriting the Control Barrier Function in Eq. (1) as a linear constraint on the control input. Specifically, A_i , defined in Eq. (7), is the row vector based on the relative position between agents i and j . The value b_i , defined in Eq. (8), is the right-hand-side safety bound that includes both the CBF term and the CVaR uncertainty term.

IV. IMPLEMENTATION DETAILS

A. Core Functions

The first two functions I constructed were `CVaR()`, which implements Eq. (2), and `h_ij()`, which implements Eq. (1), both of which were straightforward enough to code utilizing `minimize_scalar` and `numpy`. Integrating Eq. (3) was a bit more difficult due to the complexity of the CVaR uncertainty term in the equation, so I constructed a function called `CVaR_uncertainty_term()`, which I would call in my pairwise loss function `L_ij()` to divide the work and simplify the process. I then made `R_i()` to calculate the aggregated risk using `L_ij()`, `w_i()` to obtain the responsibility weight for agent i , and `A()` and `b()` to compute the linearized CBF constraints for the minimum-deviation control problem. In order to complete my function

`solve_minimum_deviation_control()`, which is responsible for implementing the minimum-deviation controller, I needed to incorporate the Python library CVXPY.

B. Scenario 1: Ramp Merging

Scenario 1, from the paper, involves taking three simulated vehicles and having them cross the ramp merging point quickly without any collisions. The evaluation is done by recording the pairwise distances between the vehicles over 50 trials and verifying if they stay above the minimum safety distance. In order for the code to fully execute, the function `run_50_trials()` is the first to be called. It then calls the function `simulate_ramp_merge()` 50 times. Before this function can be called each time, the function `sample_initial_conditions()` is called to produce random sets of x_0 and v_0 , which are the initial sets of vehicle positions and velocities, respectively. Once x_0 and v_0 are passed through a trial of `simulate_ramp_merge()`, the function then calls `nominal_accel()` to get the nominal control inputs for the three vehicles. `simulate_ramp_merge()` then performs the algorithm for the decentralized risk-aware safe control framework by calling the functions mentioned in the “Core Functions” section through a series of nested for loops. After all 50 trials are completed, `run_50_trials()` returns a dictionary object containing all distances between vehicles 1 and 2, 1 and 3, and 2 and 3, the minimum distances for each trial, the number of successful trials, the number of failed trials, and the results from the last trial. `plot_50_trial_distances()` takes the results of `run_50_trials()` and plots and saves, as a PNG file, the pairwise distances between the three vehicles over time for each of the 50 trials and displays whether or not any of those distances went below the minimum safety distance. The function `print_50_trial_summary()` prints the minimum pairwise distances, number of successful trials, and number of failed trials returned from `run_50_trials()`. The functions `plot_trajectories()`, `plot_pairwise_distances()`, and `plot_risks()` take the results of the last trial and plot and save PNG files of the three vehicles’ trajectories, pairwise distances, and risks, respectively. The function `count_scenario1_violations()` returns the number of times across the 50 trials that safety violations, going below the minimum safety distances, were made.

C. Scenario 2: Position Swapping

Scenario 2, from the paper, involves six simulated agents, with each one being tasked with swapping positions with a different agent. This simulation is performed for both the risk-aware decentralized controller and a non-risk-aware baseline controller for performance comparison. The function `simulate_pos_swap()` is similar to its Scenario 1 variant but I had to adjust the number of agents N from 3 to 6. In addition to this, I adjusted some of the other parameters, like γ , the minimum safety distance, and ϵ , for the sake

of optimization. Another key difference is that I needed to use the nominal controller that the paper prescribed for Scenario 2 by constructing the function `nominal_swap()`, which includes the right-hand heuristic needed to solve the deadlock problem. `simulate_pos_swap()` has a much larger return dictionary than its Scenario 1 counterpart as it returns the agent position history, goal positions, timestep size, safety radius, completion time, minimum pairwise distance, number of safety violations, final goal errors for each agent, aggregated risk history, responsibility-weight history, and control-deviation history. The responsibility-weight history was added for the sake of debugging because, initially, the results for the decentralized risk-aware controller and the baseline controller were too similar, so I needed to debug the process to find a solution. An important note is that `simulate_pos_swap()` can be used for both the decentralized risk-aware controller and the non-risk-aware baseline. If “True” is passed through the function, the responsibility weight will be calculated according to Eq. (5). If “False” is passed, then the responsibility weight will be fixed to 0.5. `plot_pos_swap_trajectories()`, for the paper’s framework or the baseline, would be responsible for plotting the trajectories of the six agents and saving the plot to a PNG file. The function `animate_pos_swap()` would take the agent position history, agent goals, timesteps, and the minimum safety distance in order to animate the trajectories of the six agents and display the animation as well as save it as an MP4 file. In order to accomplish this, the function needed to import `FuncAnimation` from `matplotlib.animation`. The `print_scenario2_results()` function prints the main Scenario 2 evaluation metrics for a given controller, including the completion time, minimum pairwise distance, number of safety violations, and final goal error for each agent.

V. EXPERIMENTAL RESULTS

A. Scenario 1 Results

My code recreation of the simulation for Scenario 1 showcased the majority, but not all, of the 50 trials not having any safety violations. The safety violation in this case means going under the minimum safety distance represented by the dashed line in Fig. 1.

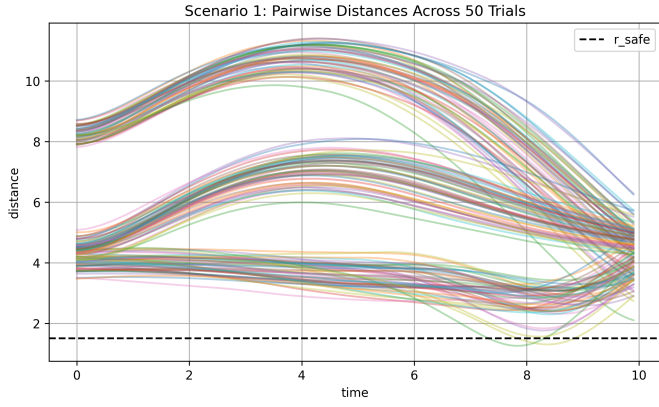


Fig. 1. Pairwise distances across 50 ramp-merging trials. The dashed line represents the minimum safe distance that each pair of vehicles are allowed to have.

The code reported that out of the 15,000 time steps during those 50 trials, there were only 20 violations of the minimum safety distance, resulting in a violation rate of approximately 0.13333 percent.

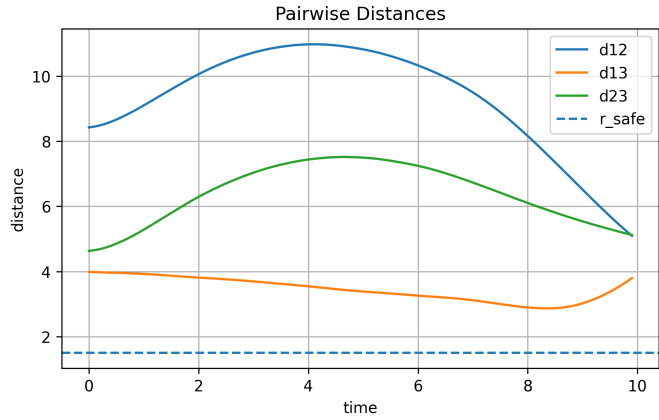


Fig. 2. Pairwise distances for a representative Scenario 1 trial. The dotted line represents the minimum safety distance.

In Fig. 2 we can see that the pairwise distances of the last trial never cross the minimum safety distance.

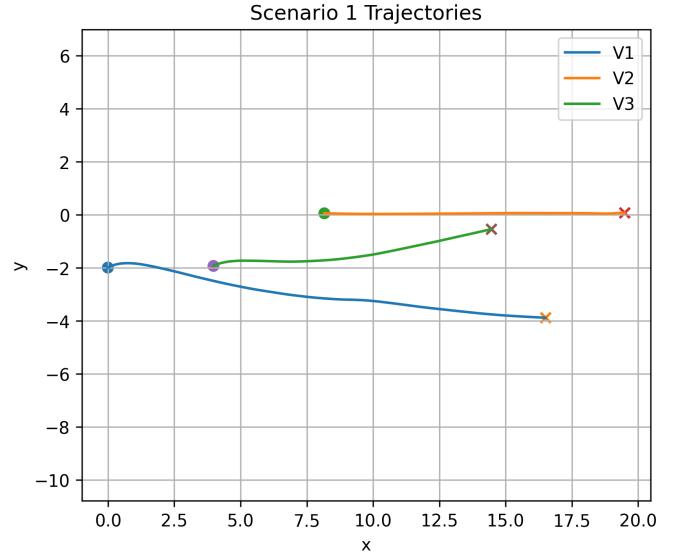


Fig. 3. Vehicle trajectories for a representative ramp-merging trial.

Fig. 3 shows the trajectories of the final trial and showcases that none of the three vehicles ever crossed paths.

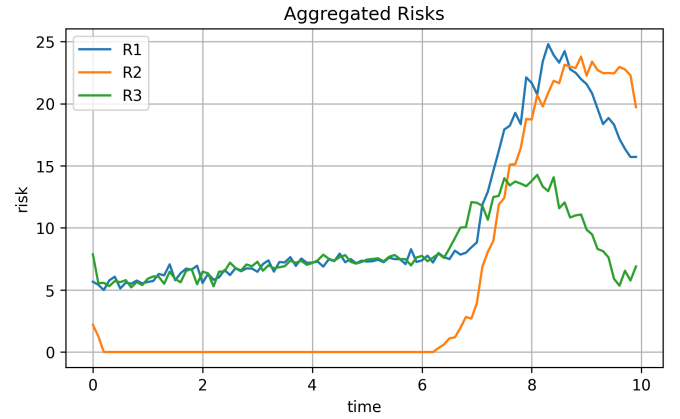


Fig. 4. Aggregated risk values for the three vehicles in a representative Scenario 1 trial.

In Fig. 4 we can see that 2 of the vehicles began with a higher risk than the third, but all three risk levels peak at around 8 seconds, likely the time they arrive at the ramp merge point. This proves that risk evaluation functions are working because the risk levels are not stagnant throughout the experiment.

Overall, the results of the ramp experiment were very optimal, but not perfect, with minimal safety violations and a success rate of 99.8667 percent. However, this differs from the original paper as the results in said paper had a safety success rate of 100 percent.

B. Scenario 2 Results

Both the decentralized risk-aware controller and the non-risk-aware baseline were successful in swapping positions of the

six agents with no collisions, with the only notable differences being that the risk-aware model required more computation time and the baseline model had a smaller minimum distance, as shown in Table I.

TABLE I
SCENARIO 2 COMPARISON BETWEEN THE RISK-AWARE CONTROLLER AND NON-RISK-AWARE BASELINE.

Controller	Time (s)	Min Dist.	Violations
Risk-aware	26.50	1.9924	0
Non-risk-aware baseline	25.30	1.7747	0

The similarities between the two models is further highlighted when comparing their similar plotted trajectories in Fig. 5 and Fig. 6.

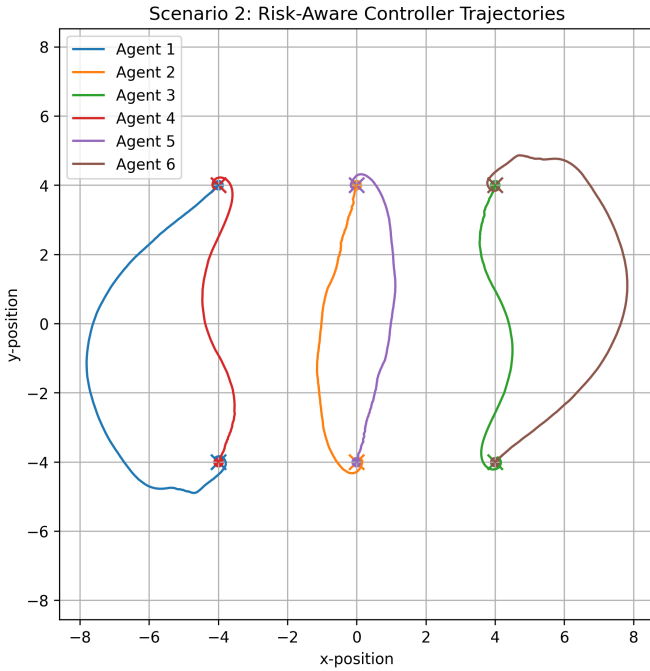


Fig. 5. Scenario 2 trajectories for the risk-aware controller.

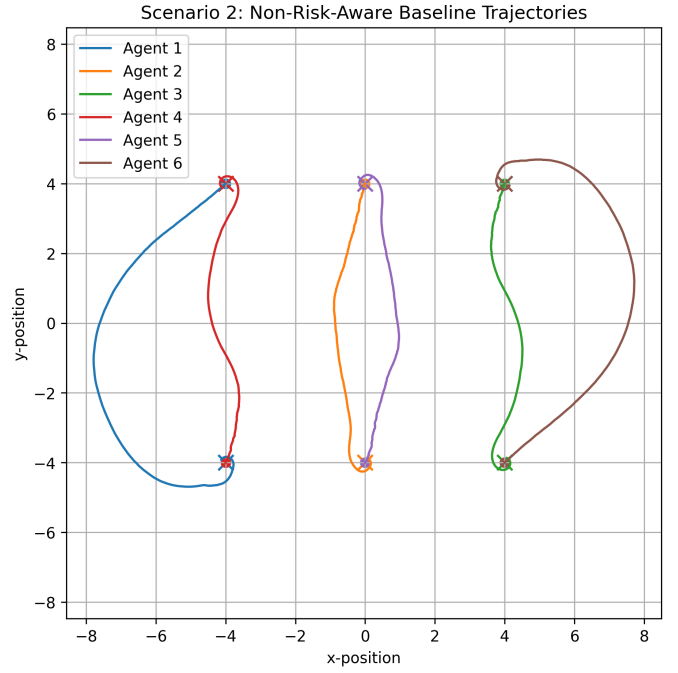


Fig. 6. Scenario 2 trajectories for the non-risk-aware fixed-weight baseline.

These results showcase a tradeoff to using the risk-aware controller and the non-risk-aware baseline. The baseline would be able to perform its task more quickly but it is at a larger risk of causing violations as it is less careful about approaching the minimum safe distance. The risk-aware controller, on the other hand, is slower but is much safer. This differs from the results in the paper because the paper's risk-aware controller was not as conservative as mine as my implementation required more time.

VI. DISCUSSION

In Scenario 1, the controller completed all 50 trials and maintained the safety margin for most pairwise timestep checks. However, a small number of violations occurred, indicating that the reproduction did not fully match the original paper's perfect safety result. In Scenario 2, both the risk-aware controller and the fixed-weight baseline completed the task without safety violations. The risk-aware controller maintained a larger minimum pairwise distance, but required a longer completion time, suggesting a more conservative behavior than the baseline.

VII. LIMITATIONS

This reproduction has several limitations. First, the original paper did not provide all implementation parameters, so values such as controller gains, uncertainty covariance, safety radius, and right-hand heuristic strength were selected manually. Second, the simulation environments were simplified compared with the original paper.

VIII. CONCLUSION

This project implemented a reproduction of the risk-aware decentralized safe-control framework proposed by Lyu, Luo, and Dolan. The implementation translated the paper’s main algorithmic components into Python, including CVaR-based uncertainty risk, pairwise Control Barrier Functions, aggregated agent risk, dynamic responsibility allocation, and the decentralized minimum-deviation controller. The reproduced controller was evaluated in two simulation scenarios: ramp merging and multi-agent position swapping. In Scenario 1, the controller completed all 50 trials and achieved a high safety-check success rate, although a small number of safety-margin violations remained. In Scenario 2, the risk-aware controller and the fixed-weight baseline both completed the task without safety violations, but the risk-aware controller maintained a larger minimum pairwise distance at the cost of longer completion time. Overall, the project successfully reproduced the main structure and qualitative behavior of the proposed framework, while also revealing implementation sensitivities related to parameter selection, uncertainty modeling, and simplified simulation design.

IX. CODE AND VIDEO AVAILABILITY

The project code is available in the GitHub repository below. The repository includes the Python implementation, requirements file, README, and saved result figures.

GitHub Repository

<https://github.com/connorh3333/CSCE-635---A.I.-for-Robotics-Final-Project---Connor-Hill.git>

A video demonstration of the Scenario 2 risk-aware controller is available through the Google Drive link below.

Scenario 2 Animation Video

<https://drive.google.com/file/d/149J6iv1JrI8KnLK0n9vb60i9wsvYoKUB/view?usp=sharing>

REFERENCES

- [1] Y. Lyu, W. Luo, and J. M. Dolan, “Risk-Aware Safe Control for Decentralized Multi-Agent Systems via Dynamic Responsibility Allocation,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023.
- [2] S. Zhang, K. Garg, C. Fan, and C. Fan, “GCBF+: A Neural Graph Control Barrier Function Framework for Distributed Safe Multi-Agent Control,” GitHub repository, MIT REALM Lab. [Online]. Available: <https://github.com/MIT-REALM/gcbfplus>
- [3] Z. Qin, K. Zhang, Y. Chen, J. Chen, and C. Fan, “Learning Safe Multi-Agent Control with Decentralized Neural Barrier Certificates,” arXiv:2101.05436, 2021.
- [4] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, “Safe Multi-Agent Interaction through Robust Control Barrier Functions with Learned Uncertainties,” in *Proc. IEEE Conference on Decision and Control (CDC)*, 2020. Code available: https://github.com/rcheng805/robust_cbf